



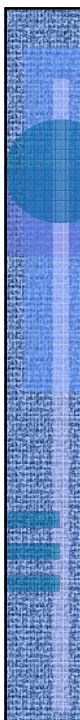
Programación Orientada a Objetos en C#

Unidad 4.- Polimorfismo

Autor:
Dr. Ramón Roque Hernández
<http://ramonroque.com/Materias/pooTec.htm>
ramonroque@yahoo.com

Colaborador:
Ing. Bruno López Takeyas, M.C.
www.itnuevolaredo.edu.mx/takeyas
takeyas@itnuevolaredo.edu.mx

1

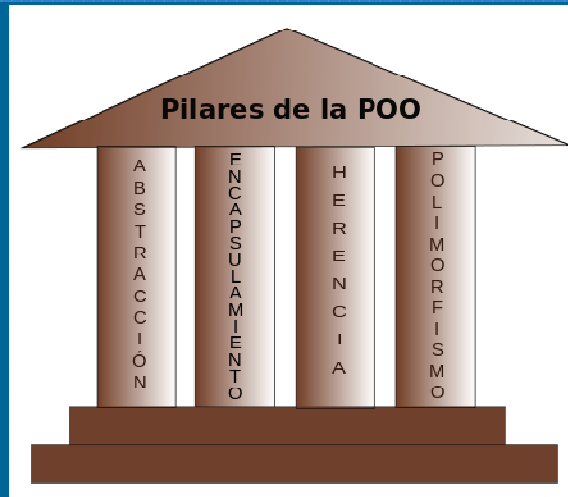


Pilares de la POO

1. Encapsulamiento
2. Abstracción
3. Herencia
4. Polimorfismo

2

Pilares de la POO



3

Polimorfismo

- Es la habilidad que poseen los objetos para reaccionar de modo diferente ante los mismos mensajes.
- El *polimorfismo* se refiere a la posibilidad de definir múltiples clases con funcionalidad diferente, pero con métodos o propiedades denominados de forma idéntica, que pueden utilizarse de manera intercambiable mediante código cliente en tiempo de ejecución.
- En C# el polimorfismo está íntimamente relacionado con la sobrecarga y métodos virtuales.

4

Conceptos relacionados con polimorfismo

- Sobrecarga (overload)
- Herencia
- Sobre-escritura (override)

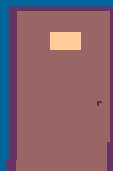
5

Sobrecarga [Overload]

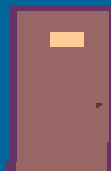
- La sobrecarga representa diferentes maneras de realizar una misma acción.
- En los programas se usa el mismo nombre en diferentes métodos con diferentes firmas [número, orden y tipo de los parámetros].
- El código de programación asociado a cada sobrecarga puede variar.
- Ejemplos:
 - `miEmpleado.Contratar("Juan", "Ventas", 2500)`
 - `miEmpleado.Contratar("Juan")`
 - `miEmpleado.Contratar("Juan", 2500)`

6

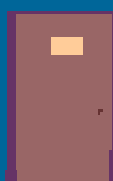
Ejemplo de Sobrecarga [Overload]



miPuerta.Abrir (Adentro, Afuera)



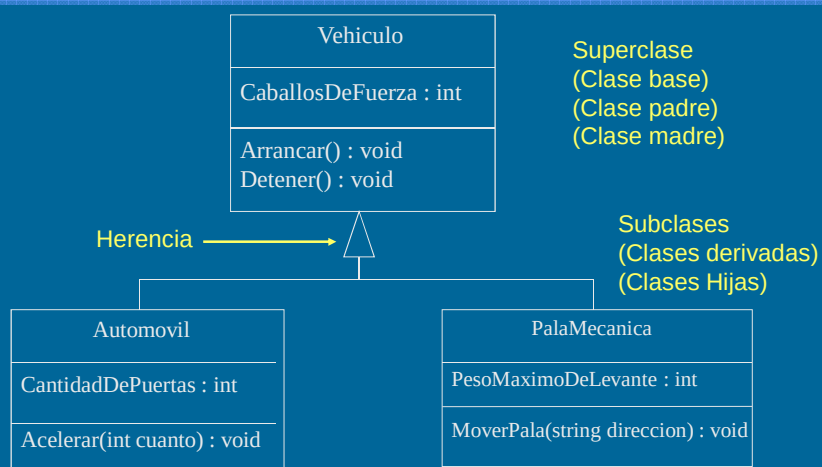
miPuerta.Abrir (Afuera, Adentro)



miPuerta.Abrir ()

7

Herencia



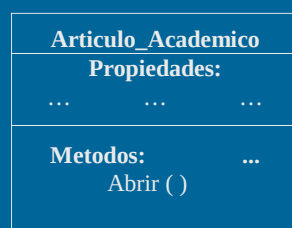
8

Anulación / Sustitución / Sobreescritura [Overriding]

- Sucede cuando una clase “B” hereda características de una clase “A”, pero la clase “B” re-define las características heredadas de “A”.
- Propiedades y métodos pueden heredarse de una superclase. Si estas propiedades y métodos son re-definidos en la clase derivada, se dice que han sido “Sobreescritos”.

9

Anulación / Sustitución / Sobreescritura [Overriding]



miLibro y **miLaptop** heredan el método **Abrir ()** pero NO lo utilizan; sino que cada uno lo implementa nuevamente de manera distinta.

Herencia



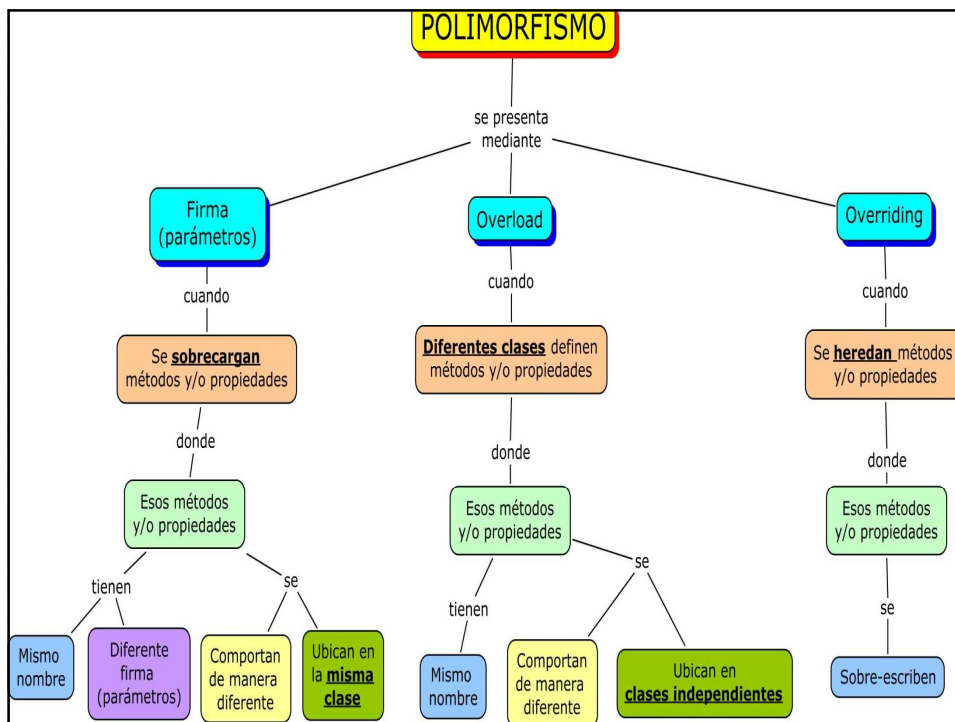
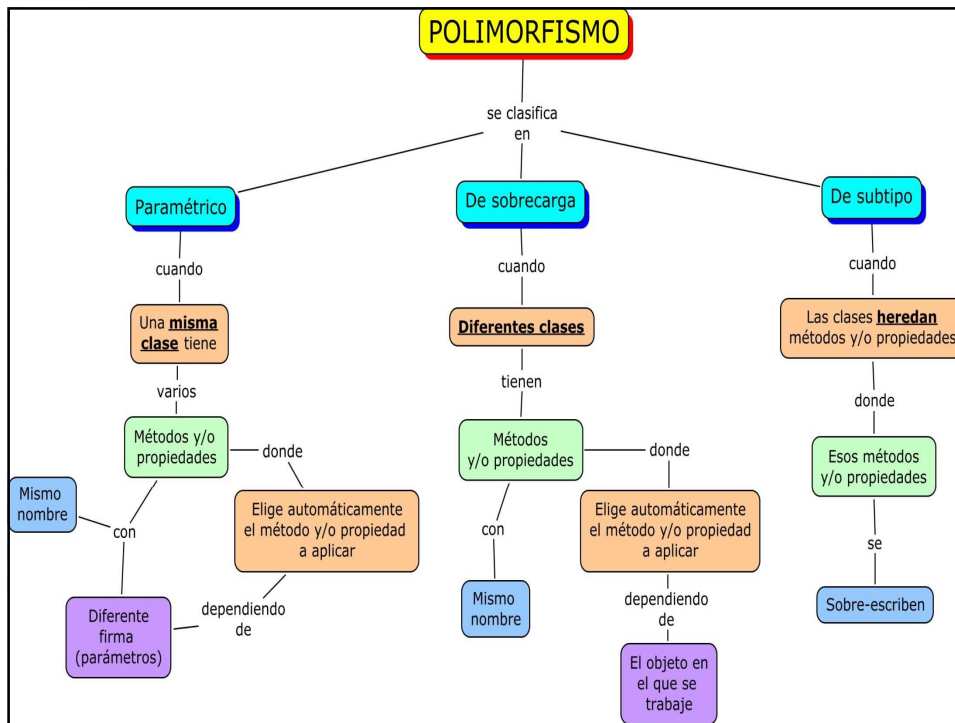
miLibro.Abrir()



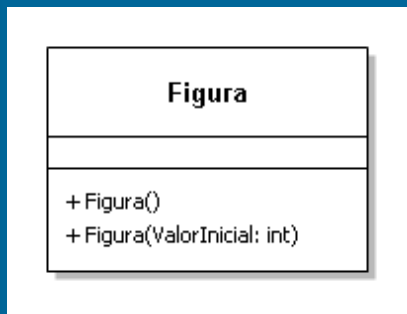
miLaptop.Abrir()

Un libro y una Laptop se abren de diferente manera.

10



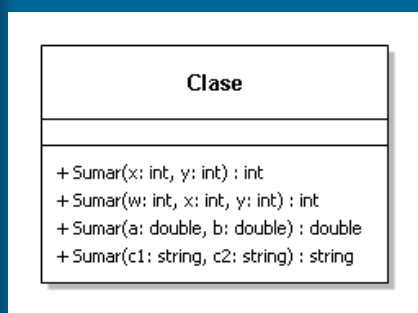
Ejemplo de polimorfismo paramétrico



- Una clase define varios métodos con el mismo nombre pero diferente firma (sobrecarga)
- Se elige el método de acuerdo a la firma aplicada
- La sobrecarga del constructor es un ejemplo de ello

13

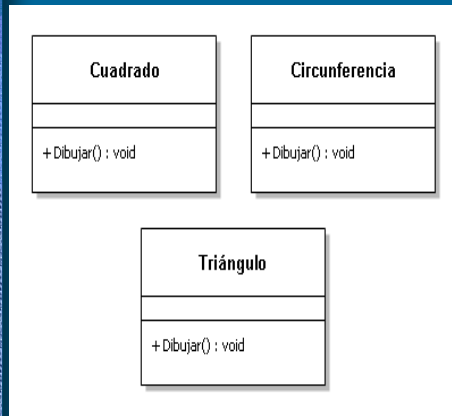
Otro ejemplo de polimorfismo paramétrico



La misma clase tiene varios métodos con el mismo nombre pero diferentes firmas con diferentes tipos de datos

14

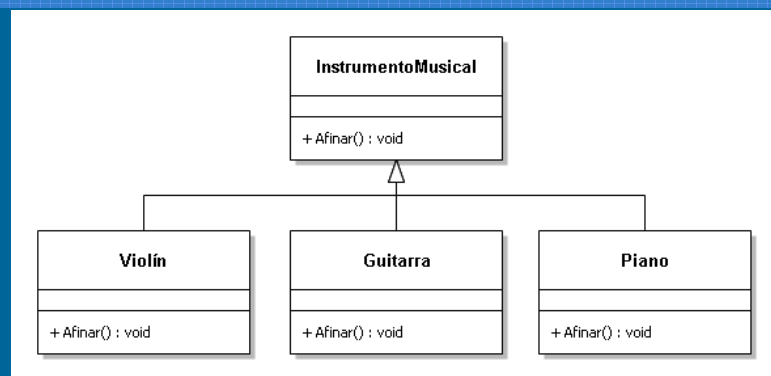
Ejemplo de polimorfismo de sobrecarga (overload)



- Diferentes clases tienen un método con el mismo nombre, pero comportamiento diferente
- Se aplica el método de acuerdo al objeto en que se trabaje

15

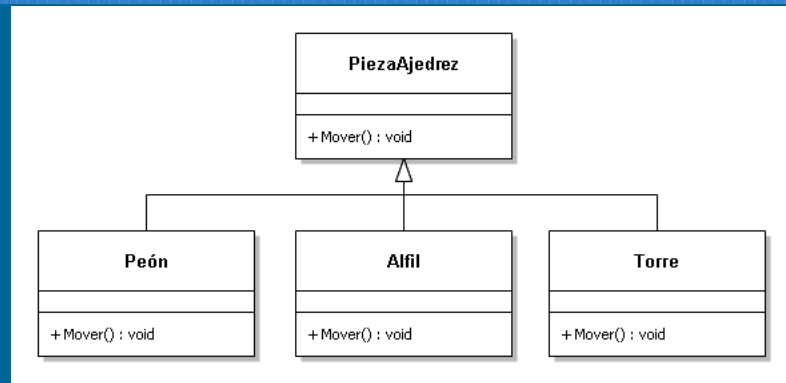
Ejemplo de polimorfismo de subtipo (override)



- Las clases derivadas redefinen los métodos y/o propiedades heredados mediante la sobre-escritura (override)

16

Otro ejemplo de polimorfismo de subtipo (override)



- Se sobre-escribe el método heredado **Mover()** según lo requiera la pieza del ajedrez

17

Diferencia entre Polimorfismo y Sobrecarga

- Un método está sobrecargado si dentro de una clase existen dos o más declaraciones de dicho método con el mismo nombre pero con parámetros distintos.
- En definitiva: La sobrecarga se resuelve en tiempo de compilación utilizando los nombres de los métodos y los tipos de sus parámetros; el polimorfismo se resuelve en tiempo de ejecución del programa, esto es, mientras se ejecuta, en función de la clase a la que pertenece el objeto.

18

Polimorfismo

POLI = Múltiples **MORFISMO** = Formas



miRefrigerador.Abrir("Puerta de Abajo")
miRefrigerador.Abrir("Puerta de Arriba", "Mitad")

ObjetoEnFormaDeCaja



miRegalo.Abrir()



miCofre.Abrir()

19

Métodos Virtuales

- Son métodos en la clase base pensados para ser sobre-escritos por subclases.
- Para declararlos, se utiliza la palabra reservada "virtual"; para sobre-escribirlos, en la subclase se utiliza la palabra reservada "override".
- Un método virtual "PUEDE" ser sobreescrito, o utilizarse tal como está.
- Solo se puede utilizar "override" si el método en la clase base está marcado como "virtual", "abstract" u "override".
- El método "override" debe mantener el mismo nivel de acceso que el método "Virtual" correspondiente

20

Ejemplo virtual...override

```
class Vehiculo
{
    public virtual void Arrancar()
    {
        System.Console.WriteLine("Arrancar...Clase Vehiculo");
    }
}

class Carro: Vehiculo
{
    public override void Arrancar()
    {
        System.Console.WriteLine("Arrancar....Clase Carro");
    }
}

class Programa
{
    static void Main()
    {
        Carro miCarro = new Carro();
        miCarro.Arrancar();
        System.Console.ReadLine();
    }
}
```

Ejecución del programa...
Arrancar....Clase Carro

21

Ejemplo virtual...override (Polimorfismo en Tiempo de ejecución)

```
class Vehiculo
{
    public virtual void Arrancar()
    {
        System.Console.WriteLine("Arrancar...Clase Vehiculo");
    }
}

class Carro : Vehiculo
{
    public override void Arrancar()
    {
        System.Console.WriteLine("Arrancar....Clase Carro");
    }
}

class Programa
{
    static void Main()
    {
        Vehiculo v;
        v = new Vehiculo();
        v.Arrancar();
        v = new Carro();
        v.Arrancar();
        System.Console.ReadLine();
    }
}
```

En una variable tipo "Vehiculo" se almacenan objetos tipo "Vehiculo" y tipo "Carro".
Al invocar el mismo método para el mismo objeto, se observa una conducta diferente, apropiada para cada objeto.

Ejecución del programa...
Arrancar...Clase Vehiculo
Arrancar....Clase Carro

22

override sealed

- Agregar “sealed” a un método “override” impide la futura sobre-escritura de ese método, proporcionando una implementación final.

```
class Aparato
{
    public virtual void Prender()
    {
        System.Console.WriteLine(" Prendiendo el Aparato ");
    }
}

class TV : Aparato
{
    public override sealed void Prender()
    {
        System.Console.WriteLine(" LA TV SE ESTA PRENDIENDO ");
    }
}

class TVColor : TV
{
    public override void Prender()
    {
        System.Console.WriteLine(" La tele a color se esta prendiendo");
    }
}
```

ERROR!!!!...El método ya no se puede sobrescribir.

23

Ocultar métodos heredados

- Es posible ocultar un método heredado e introducir uno nuevo a la jerarquía de clases. El método antiguo (heredado) es reemplazado por otro nuevo, diferente, pero con el mismo nombre y la misma firma.

```
class Vehiculo
{
    public void Arrancar()
    {
        System.Console.WriteLine(" Clase Vehiculo. Metodo Arrancar ");
    }
}

class Automovil : Vehiculo
{
    public new void Arrancar()
    {
        System.Console.WriteLine(" Clase Automovil. Metodo Arrancar ");
    }
}
```

24

Referencias

- <http://www.devjoker.com/contenidos/Tutorial-C/135/Polimorfismo.aspx>
- <http://www.nebrija.es/~abustind/Informatica/MetodologiaII/Polimorfismo.pdf>
- <http://apdaza-poo.blogspot.mx/2008/04/polimorfismo.html>
- <http://www.clikear.com/manuales/csharp/c38.aspx>
- http://www.elguille.info/colabora/NET2005/jbarrientos_polimorfismoCS.htm

25